Audio Signals, Alignment, and Speech Recognition Objectives

Mathematics of Deep Learning — Chapter 19

Yin Yang

Foundation Books Project

Roadmap

Audio to Discrete Signal

Frames and Log-Mel Features

Tokens and Normalization

The Alignment Problem

Framewise Classification

Connectionist Temporal Classification

Neural Encoders

Attention-Based ASR

Transducers and Streaming

Decoding and Language Models

Evaluation: WER and CER

Robustness and Deployment

Summary

Audio to Discrete Signal

The Core ASR Problem

Automatic speech recognition maps an acoustic signal to text. Running example:

"turn on the kitchen light".

Mathematically,

$$x_{1:T} \longmapsto y_{1:U},$$

where $x_{1:T}$ is a long sequence of audio frames and $y_{1:U}$ is a shorter sequence of transcript tokens.

- T: hundreds of audio frames.
- U: a handful of words (or word pieces, or characters).
- The mismatch $T \gg U$ drives every objective in this chapter.

ASR is a sequence problem where the time alignment between audio and text is hidden.

Waveforms and Sampling Rate

A mono digital waveform is a finite vector

$$s = (s_0, s_1, \ldots, s_{N-1}) \in \mathbb{R}^N,$$

recorded at sampling rate F_s samples per second. Duration:

$$\text{duration}(s) = \frac{N}{F_s}.$$

Example: 2.5 seconds at 16,000 Hz gives $N = 40{,}000$ samples.

- Nyquist: $F_s > 2B$ to capture content below B Hz.
- Energy above $F_s/2$ aliases into low frequencies.
- A batch is a matrix $S \in \mathbb{R}^{m \times N_{\max}}$ with a length vector $n \in \mathbb{N}^m$; padded entries must be masked.

Keep samples, seconds, and length masks travelling together — the wrong sampling-rate label silently changes time.

Frames and Log-Mel Features

From Waveform to Feature Matrix

waveform $s \in \mathbb{R}^N$

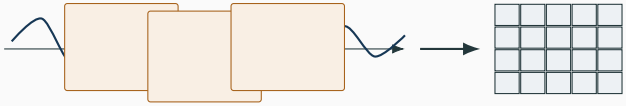
(1) Sampled waveform changes fast.

From Waveform to Feature Matrix

waveform $s \in \mathbb{R}^N$
overlapping frames: length L, hop H

(1) Sampled waveform changes fast. **(2)** Frames smooth a short window of samples.

From Waveform to Feature Matrix

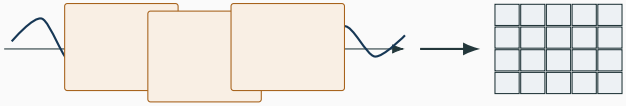


waveform $s \in \mathbb{R}^N$
overlapping frames: length L, hop H

feature matrix
$x \in \mathbb{R}^{T \times B}$

(1) Sampled waveform changes fast. **(2)** Frames smooth a short window of samples. **(3)** Each frame becomes a feature row.

From Waveform to Feature Matrix



waveform $s \in \mathbb{R}^N$
overlapping frames: length L, hop H

feature matrix
$x \in \mathbb{R}^{T \times B}$

$T = 1 + \left\lfloor \frac{N-L}{H} \right\rfloor$
$L = 400,\ H = 160,\ N = 28800$
$\Rightarrow\ T = 178.$
With $B = 80$ mel bands: 178×80.

(1) Sampled waveform changes fast. **(2)** Frames smooth a short window of samples. **(3)** Each frame becomes a feature row. **(4)** Frame count T is the model's time axis.

Frame length and hop set the model's sequence length, encoder cost, and latency.

Spectrograms and Log-Mel Features

Short-time Fourier transform magnitude at frame t and bin k:

$$X_{t,k} = \left| \sum_{n=0}^{L-1} s_{tH+n}\, w_n\, e^{-2\pi i k n / L} \right|.$$

Mel-energy compression with filter bank $M \in \mathbb{R}_{\geq 0}^{B \times K}$:

$$E_{t,b} = \sum_{k=1}^{K} M_{b,k}\, X_{t,k}^2, \qquad x_{t,b} = \log(E_{t,b} + \epsilon).$$

- Spectrogram $X \in \mathbb{R}_{\geq 0}^{T \times K}$: nonnegative time-by-frequency.
- Log-mel features $x \in \mathbb{R}^{T \times B}$ compress frequency and stabilize scale.
- $\epsilon > 0$ prevents $\log 0$ when energies are near zero.

Record the feature recipe and use the *same* recipe at training and deployment.

Tokens and Normalization

Transcripts, Vocabularies, Normalization

A vocabulary $\mathcal{V}$ is a finite set of output symbols. A transcript is

$$y_{1:U} = (y_1, \ldots, y_U), \qquad y_u \in \mathcal{V}.$$

Same utterance, different vocabularies:

- Words: `(turn, on, the, kitchen, light)` — length 5.
- Characters: `t,u,r,n,␣,o,n,␣,t,h,e,␣,k,...,t` — length 25.
- Word pieces (BPE): common words intact, rare words split.

Normalization is a deterministic policy: lower-case, punctuation, numbers, fillers. *Reference* and *hypothesis* are only comparable under a stated policy.

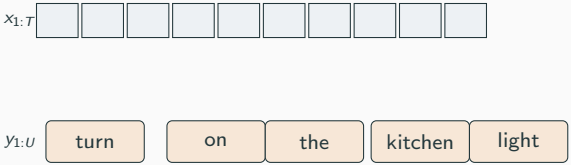
Tokenization decides what the recognizer is allowed to spell — report WER together with its normalization rule.

The Alignment Problem

The Hidden Alignment Problem

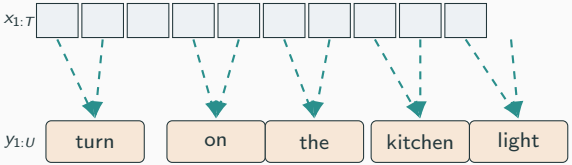
(1) Many frames carry the acoustic signal.

The Hidden Alignment Problem



(1) Many frames carry the acoustic signal.
(2) Few tokens carry the transcript.

The Hidden Alignment Problem

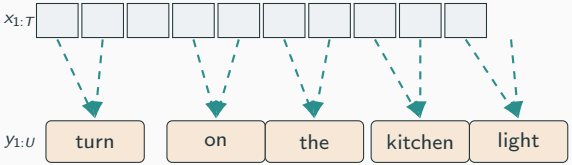


(1) Many frames carry the acoustic signal.

(2) Few tokens carry the transcript.

(3) A frame-level alignment $a_{1:T} \in (\mathcal{V}')^T$ assigns one label per frame.

The Hidden Alignment Problem



Latent alignment
training data gives only $y_{1:U}$, not the path that picks which frames support which token.

(1) Many frames carry the acoustic signal.
(2) Few tokens carry the transcript.
(3) A frame-level alignment $a_{1:T} \in (\mathcal{V}')^T$ assigns one label per frame.
(4) Alignment is usually *not* provided.

Latent-alignment objectives sum over many compatible paths instead of guessing one.

Framewise Classification

Framewise Cross-Entropy: Warm-Up

Assume *known* frame labels $a_{1:T} \in (\mathcal{V}')^T$. The encoder gives per-frame probabilities

$$p_\theta(c \mid x, t) = \mathrm{softmax}(z_t)_c, \qquad c \in \mathcal{V}'.$$

Framewise loss:

$$\mathcal{L}_{\mathrm{frame}}(\theta) = -\sum_{t=1}^{T} \log p_\theta(a_t \mid x, t).$$

Three-frame example with correct-label probabilities $0.8, 0.6, 0.5$:

$$-\log 0.8 - \log 0.6 - \log 0.5 \approx 0.22 + 0.51 + 0.69 = 1.42.$$

Masked-out padded frames contribute 0.

Easy when every frame has a target — but ordinary transcripts do not supply a_t.

Connectionist Temporal Classification

CTC: Blanks, Paths, Collapse Map

Extend the vocabulary with a blank $\varnothing \notin \mathcal{V}$:

$$\mathcal{V}' = \mathcal{V} \cup \{\varnothing\}.$$

A CTC path is $\pi_{1:T} \in (\mathcal{V}')^T$. The collapse map B:

1. merge consecutive repeated nonblank symbols,
2. delete all blanks.

Examples:

$$B(\varnothing, a, a, \varnothing, b, b) = ab, \qquad B(a, \varnothing, a) = aa.$$

Conditional-independence factorization and transcript probability:

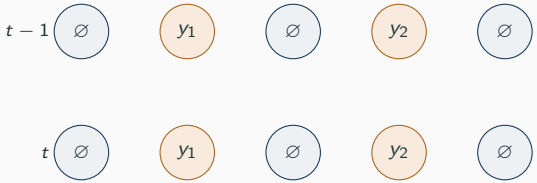
$$p_\theta(\pi \mid x) = \prod_{t=1}^{T} p_\theta(\pi_t \mid x, t), \quad p_\theta(y \mid x) = \sum_{\pi \in B^{-1}(y)} p_\theta(\pi \mid x).$$

Blanks let the path *wait*, and they are what separate repeated output symbols.

CTC Forward Lattice: Stay, Advance, Skip

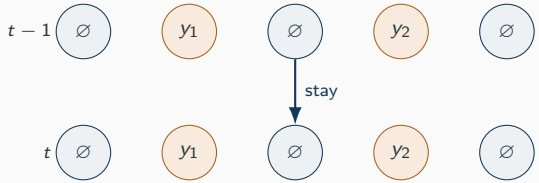
(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.

CTC Forward Lattice: Stay, Advance, Skip



(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.
(2) Same row at the next time step.

CTC Forward Lattice: Stay, Advance, Skip

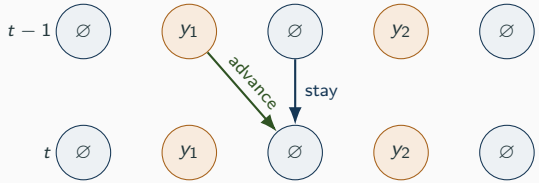


(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.

(2) Same row at the next time step.

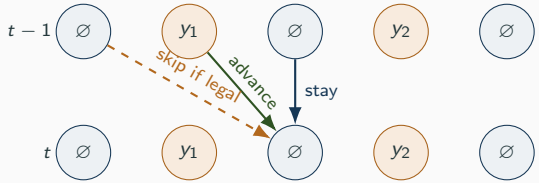
(3) Stay in place: same expanded position.

CTC Forward Lattice: Stay, Advance, Skip



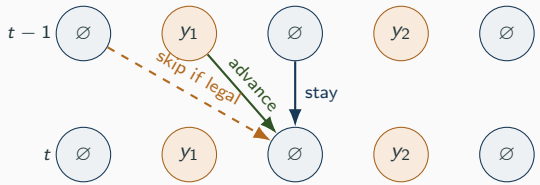
(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.
(2) Same row at the next time step.
(3) Stay in place: same expanded position.
(4) Advance by one position.

CTC Forward Lattice: Stay, Advance, Skip



(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.
(2) Same row at the next time step.
(3) Stay in place: same expanded position.
(4) Advance by one position.
(5) Skip a blank when the surrounding labels differ.

CTC Forward Lattice: Stay, Advance, Skip



$\alpha_t(r) = p_\theta(\ell_r \mid x, t) \cdot$
$[\alpha_{t-1}(r) + \alpha_{t-1}(r-1) + q_r \alpha_{t-1}(r-2)]$
skip legal iff $\ell_r \neq \varnothing$ and $\ell_r \neq \ell_{r-2}$.

(1) Expanded labels $\ell = (\varnothing, y_1, \varnothing, y_2, \varnothing)$.
(2) Same row at the next time step.
(3) Stay in place: same expanded position.
(4) Advance by one position.
(5) Skip a blank when the surrounding labels differ.
(6) Add predecessor mass, multiply by frame probability.

Dynamic programming sums all valid frame paths in $O(T \cdot 2U)$ time.

CTC Loss, Posteriors, Log-Domain

CTC loss for one example:

$$\mathcal{L}_{\mathrm{CTC}}(\theta) = -\log p_\theta(y \mid x) = -\log\big[\alpha_T(2U) + \alpha_T(2U+1)\big].$$

State posteriors via backward variables $\beta_t(r)$ and $Z = p_\theta(y \mid x)$:

$$\gamma_t(r) = \frac{\alpha_t(r)\beta_t(r)}{Z}, \qquad \Gamma_t(c) = \sum_{r:\ell_r=c} \gamma_t(r).$$

Softmax-logit gradient has the familiar form

$$\frac{\partial \mathcal{L}_{\mathrm{CTC}}}{\partial z_{t,c}} = p_t(c) - \Gamma_t(c).$$

Log-domain recurrence (numerical safety):

$$g_t(r) = \log p_\theta(\ell_r \mid x, t) + \mathrm{logsumexp}\big\{g_{t-1}(r),\ g_{t-1}(r-1),\ g_{t-1}(r-2) + \log q_r\big\}.$$

Training raises logits where posterior responsibility exceeds the current probability.

Neural Encoders

Neural Encoders for Speech

An encoder maps audio features to hidden states:

$$f_\theta : \mathbb{R}^{T \times B} \to \mathbb{R}^{T' \times d}, \qquad h_{1:T'} = f_\theta(x_{1:T}).$$

Subsampling factor $\approx T/T'$ trades resolution for compute.

- **Convolutions** near the input exploit local time-frequency structure.
- **Recurrent** layers process time sequentially (early end-to-end ASR).
- **Transformers** use self-attention for global context.
- **Conformer** combines attention with convolutions.

Streaming constraint: h_t depends only on past frames plus a bounded amount of future context. The bound is the recognition latency.

Encoder choice fixes both the representation dimension d and the time resolution T'.

Attention-Based ASR

Attention Encoder-Decoder ASR

Autoregressive model over the transcript:

$$p_\theta(y_{1:U} \mid x_{1:T}) = \prod_{u=1}^{U} p_\theta(y_u \mid y_{<u}, h_{1:T'}).$$

Cross-attention with decoder state s_u:

$$e_{u,t} = s_u^\top W h_t, \quad a_{u,t} = \frac{\exp(e_{u,t})}{\sum_{t'} \exp(e_{u,t'})}, \quad c_u = \sum_{t=1}^{T'} a_{u,t}\, h_t.$$

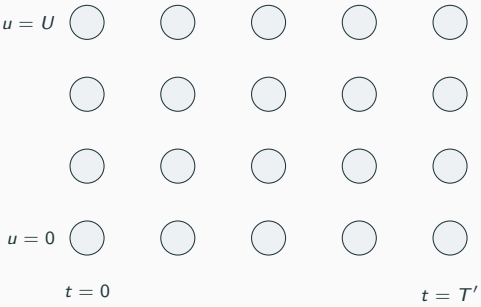
Training loss (teacher forcing):

$$\mathcal{L}_{\text{attn}} = -\sum_{u=1}^{U} \log p_\theta(y_u \mid y_{<u}, h_{1:T'}).$$

- Attention weights $a_{u,t}$ act as a *soft* alignment.
- Strong language context from $y_{<u}$.
- Risk: fluent text weakly supported by the audio.

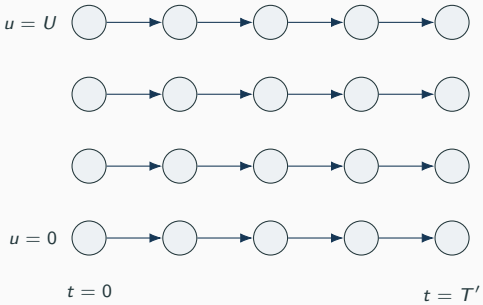
Transducers and Streaming

Transducer Grid: Time and Output Index



(1) Grid indexed by acoustic time t and output index u.

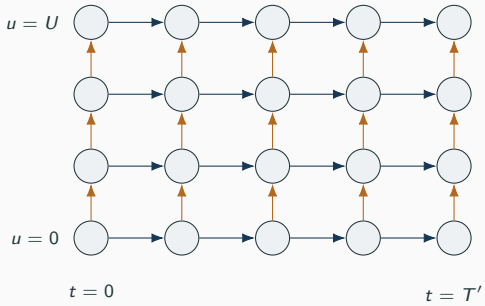
Transducer Grid: Time and Output Index



(1) Grid indexed by acoustic time t and output index u.

(2) Blue: blank move consumes one acoustic step.

Transducer Grid: Time and Output Index

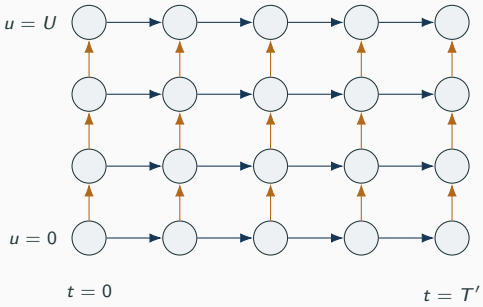


(1) Grid indexed by acoustic time t and output index u.

(2) Blue: blank move consumes one acoustic step.

(3) Orange: label move emits y_{u+1}.

Transducer Grid: Time and Output Index



Forward recurrence
$\alpha(t,u) = \alpha(t-1,u)\, p_{\varnothing}(t-1,u)$
$\quad +\alpha(t,u-1)\, p_y(t,u-1).$

Every complete path has T' blanks and U label moves.

(1) Grid indexed by acoustic time t and output index u.
(2) Blue: blank move consumes one acoustic step.
(3) Orange: label move emits y_{u+1}.
(4) Sum over all paths from $(0,0)$ to (T',U).

Transducers add an output index u on top of CTC's time axis — so token predictions can

Streaming Recognition Constraint

Streaming recognition emits partial transcripts *before* the utterance ends. The model must decide using only:

- all past frames $x_{1:t}$,
- a bounded amount of future context $x_{t+1:t+\Delta}$.

Trade-off in the transducer grid:

- Too many blank transitions $\Rightarrow$ latency or deletions.
- Too many label transitions $\Rightarrow$ output before evidence.

Design questions reduce to shape questions:

$$T,\ T',\ \Delta,\ \text{which } h_t \text{ depends on which } x_s,\ m_t \in \{0,1\}.$$

Streaming changes the *mathematical* dependency graph between audio and outputs, not just the runtime budget.

Decoding and Language Models

Beam Search and LM Fusion

Greedy CTC: $\hat{\pi}_t = \arg\max_c p_\theta(c \mid x, t)$, then return $B(\hat{\pi})$. Often misses the most likely transcript. CTC prefix beam search tracks *two* scores per prefix p:

$$P_b(p, t), \quad P_{\mathrm{nb}}(p, t) \ \text{(blank- vs. nonblank-ending mass)}.$$

The split is needed: aa and $a\varnothing a$ collapse differently. Shallow LM fusion picks

$$\hat{y} = \arg\max_y \big[\log p_{\mathrm{ASR}}(y \mid x) + \lambda \log p_{\mathrm{LM}}(y) + \alpha_{\mathrm{len}}|y|\big].$$

Decoding is part of the model: beam width, λ, and length penalty all change reported WER.

A Decoding Trade-Off

Audio: "kitchen light." Two candidates, similar acoustic scores:

Candidate	$\log p_{\mathrm{ASR}}$	$\log p_{\mathrm{LM}}$
kitchen light	-2.0	-3.0
catching light	-2.3	-1.0

With $\lambda = 0.2$:

$$\text{kitchen light: } -2.0 + 0.2(-3.0) = -2.6, \quad \text{catching light: } -2.3 + 0.2(-1.0) = -2.5.$$

The LM flips the decision: more fluent, less faithful to the audio.

An LM may improve grammar at the cost of acoustically correct rare terms — always state the decoding recipe.

Evaluation: WER and CER

Edit Distance, WER, CER

For reference $y_{1:U}$ and hypothesis $\hat{y}_{1:V}$, $D(i,j)$ is the minimum number of insertions, deletions, and substitutions:

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1, \\ D(i,j-1) + 1, \\ D(i-1,j-1) + \mathbf{1}\{y_i \neq \hat{y}_j\}, \end{cases}$$

with $D(0,j) = j$, $D(i,0) = i$. Then

$$\mathrm{WER} = \frac{S + D + I}{N}, \qquad \mathrm{CER} = \frac{S_{\mathrm{char}} + D_{\mathrm{char}} + I_{\mathrm{char}}}{N_{\mathrm{char}}}.$$

Example. Reference "turn on the kitchen light" ($N = 5$); hypothesis "turn the kitchen lights." One deletion (`on`), one substitution (`light`→`lights`):

$$\mathrm{WER} = \frac{1 + 1 + 0}{5} = 0.40.$$

WER is meaningful only together with its normalization and decoding policy.

Beyond Average WER

WER alone hides important behaviour:

- **Calibration:** are confidences trustworthy at the value p?
- **Timestamps:** a transcript with low WER can still have poor alignment.
- **Streaming latency:** partial-result delay matters to users.
- **Subgroup WER:** per-environment, per-accent, per-device.

Report metrics matching the user experience: final WER, partial-result latency, timestamp accuracy, confidence quality, subgroup error rates.

Average WER summarises a population — it does not describe what any single user hears.

Robustness and Deployment

Robustness and Domain Shift

Acoustic domain shift:

$$P_{\text{train}}(x, y) \neq P_{\text{deploy}}(x, y).$$

Common shift axes:

- Microphones, rooms, far-field vs. close-talking.
- Background noise, reverberation, channel compression.
- Accents, speaking rate, age, code-switching.
- New product names, medical terms, unusual numbers (label-domain shift).

Pitfalls: data leakage of speakers across train/test; an overly strong LM replacing rare correct words with common phrases; clean-speech benchmarks overstating deployment performance.

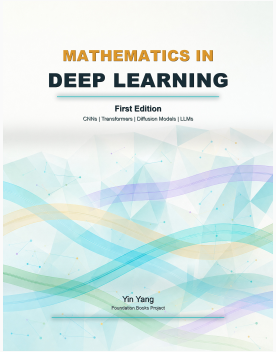
Self-supervised and weakly-supervised pretraining help — they do not replace task-specific subgroup evaluation.

Summary

Chapter in One Slide

- **Signal chain:** waveform $\rightarrow$ frames $\rightarrow$ log-mel features $x \in \mathbb{R}^{T \times B}$.
- **Tokens:** words, characters, or word pieces, under a stated normalization policy.
- **Alignment:** frame-by-token correspondence is *latent* in ordinary training data.
- **CTC:** blanks plus collapse map; loss is $-\log \sum_{\pi} p_\theta(\pi \mid x)$; dynamic programming with stay/advance/skip.
- **Attention encoder-decoder:** autoregressive transcript model; attention weights act as soft alignment.
- **Transducers:** (t, u) grid; blank moves time, label moves output; natural for streaming.
- **Decoding:** greedy vs. prefix beam vs. LM fusion — the recipe is part of the model.
- **Evaluation:** edit-distance WER/CER, plus calibration, latency, timestamps, subgroup error.

Next: bringing these objectives to multimodal and large-scale models, where audio meets text, vision, and downstream tasks.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/mathematics-in-deep-learning`
Code	`github.com/foundation-books/mathematics-in-deep-learning-code`
Paperback	`amazon.com/dp/B0H1ZZHDT2`
Hardcover	`amazon.com/dp/B0H1ZL11MJ`
Kindle	`amazon.com/dp/B0GX32M8SP`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.